SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **NAME:** S SUNDARESAN

COURSE CODE: CSA4303 **REG NO:**192411245

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.
2. Modify the structure to make it standards-compliant.
3. Interpret the HTTP request generated during form submission.
4. Analyze the HTTP response cycle from server to browser.
5. Suggest improvements for usability and accessibility.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
  <title>Company</title>
 </head>
 <body>
  <div>
   <h1>Welcome
   <p>About us
  </div>
 </body>
</html>
```

Questions:

1. Identify improper nesting and missing closing tags.
2. Analyze how different browsers may interpret this structure.
3. Identify missing semantic elements (e.g., header, section).
4. Provide a corrected W3C-compliant HTML structure.
5. Suggest techniques to ensure cross-browser compatibility.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the form configuration.
2. Analyze why the HTTP request is not triggered.
3. Interpret the role of GET method in this scenario.
4. Propose a corrected implementation.
5. Suggest improvements for secure data transmission.

Code of Conduct:

I S SUNDARAESAN /192411245 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	25
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	25
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	20
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	10
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	8
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	4

Question 1: Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure

HTML5 semantic elements describe the purpose of different parts of a webpage, improving readability, accessibility, and maintainability.

The given form lacks a meaningful page structure and accessible labels.

- <header> – Contains the symposium title and introductory information.
- <nav> – Provides navigation links to the homepage, events, and registration page.
- <main> – Contains the primary content of the webpage.
- <section> – Groups related content, such as symposium details and registration.
- <footer> – Contains contact details and copyright information.
- <label> – Provides descriptive labels for form inputs.

These elements improve document structure and help screen readers understand the page.

2. Modify the structure to make it standards-compliant

The following HTML5 code provides a valid, semantic, and accessible registration form.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>
  <header>
    <h1>Online Symposium 2026</h1>
    <p>Register to participate in our college symposium.</p>
  </header>

  <main>
    <section aria-labelledby="registration-title">
      <h2 id="registration-title">Student Registration</h2>
```

```
        <form action="/register" method="POST">
          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name"
                 autocomplete="name" required>

          <br><br>

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email"
                 autocomplete="email" required>

          <br><br>

          <button type="submit">Register Now</button>
        </form>
      </section>
    </main>

    <footer>
      <p>&copy; 2026 College Symposium</p>
    </footer>
</body>
</html>
```

Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- lang="en" identifies the document language.
- <main> and <section> organize the main content.
- <label> associates descriptive text with each input.
- required prevents submission when a mandatory field is empty.
- type="email" enables browser-based email-format validation.
- method="POST" sends form data in the HTTP request body.

The code follows standard HTML5 structure and improves accessibility.

3. Interpret the HTTP request generated during form submission

HTTP (Hypertext Transfer Protocol) enables communication between a web browser and a web server.

When a student submits the registration form, the browser generates an HTTP POST request.

Example:

POST /register HTTP/1.1
Host: symposium.example.com
Content-Type: application/x-www-form-urlencoded

name=Arun&email=arun%40example.com
Interpretation:

POST specifies the HTTP method used to submit data.

/register identifies the server resource that processes registration.

Host identifies the destination server.

Content-Type indicates the format of the submitted data.

The request body contains the student's name and email address.

Since POST is used, the form data is transmitted in the request body instead of being appended to the URL query string.

However, POST alone does not encrypt information. HTTPS is required to protect data during transmission.

4. Analyze the HTTP response cycle from server to browser

The HTTP response cycle consists of the following steps:

1. Student submits the form

Browser validates required fields

2. HTTP POST request

Registration data reaches the server

3. Server processes the request

Validates data and stores registration if successful

4. HTTP response is returned

Status code, headers and response content

5. Browser displays the result

Success message or appropriate error

Example successful response:

HTTP/1.1 201 Created
Content-Type: text/html; charset=UTF-8

The server may return a confirmation page stating that registration was successful.

Important response codes include:

- 201 Created – A new registration was successfully created.
- 400 Bad Request – The submitted request is invalid.
- 422 Unprocessable Content – The server cannot process the submitted data because of validation errors.
- 500 Internal Server Error – An unexpected server-side error occurred.

Thus, the browser-server response cycle ensures that students receive feedback about their registration.

5. Suggest improvements for usability and accessibility

The portal can be improved using the following techniques:

Meaningful labels: Associate every input with a <label> element to support screen readers.

Form validation: Use required, type="email", and suitable validation messages.

Keyboard accessibility: Ensure all controls can be accessed using the keyboard.

Responsive design: Use CSS media queries and the viewport meta tag for mobile compatibility.

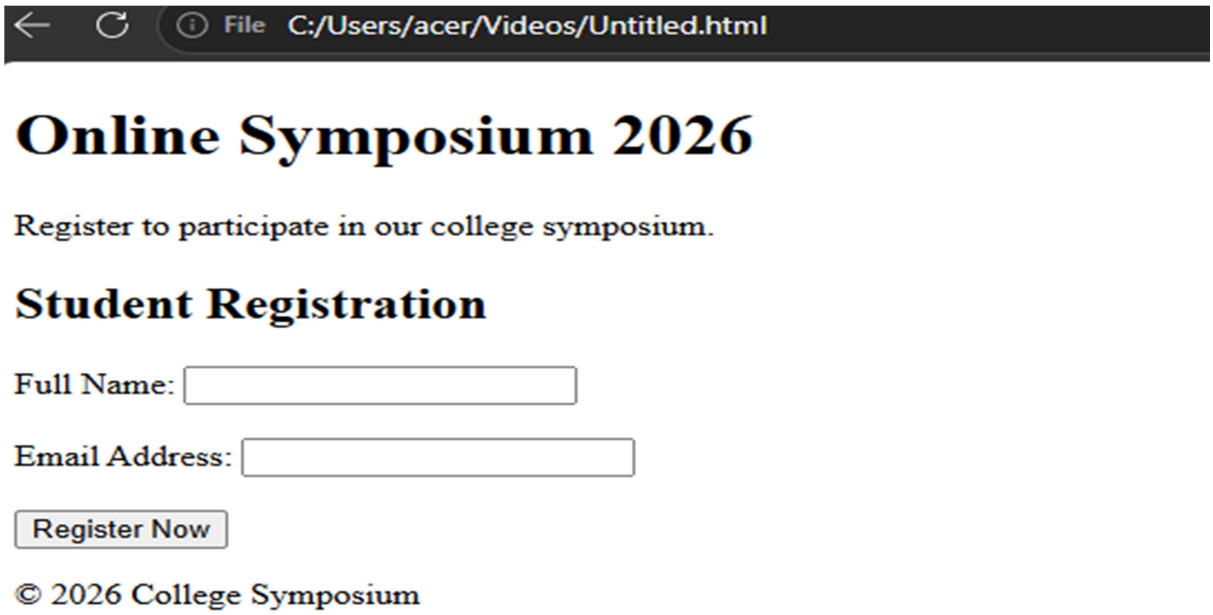
Clear feedback: Display registration confirmation and useful error messages.

Secure communication: Use HTTPS and validate submitted data on the server.

Data privacy: Collect only necessary personal information and protect stored records.

Conclusion: A semantic HTML5 form combined with HTTP communication, validation, accessibility, and HTTPS provides a reliable and user-friendly symposium registration portal.

OUTPUT:



Question 2: Cross-Browser Compatibility Issue

1.Identify improper nesting and missing closing tags

The given HTML snippet is:

```
<html>
 <head>
  <title>Company</title>
 </head>
 <body>
  <div>
   <h1>Welcome
```

```
    <p>About us

  </div>

 </body>

</html>
```

The following issues are present:

- The HTML5 doctype declaration is missing.
- The <html> element does not specify the document language.
- The <meta charset="UTF-8"> declaration is missing.
- The <h1> element has no explicit closing </h1> tag.
- The element has no explicit closing tag.
- The generic does not communicate the purpose of the content.
- Semantic elements such as <header>, <main>, and <section> are missing.

The unclosed heading and paragraph make the markup incomplete and reduce maintainability.

2 Analyze how different browsers may interpret this structure

Browsers use an HTML parsing algorithm to recover from certain markup errors.

- Some missing end tags may be implicitly handled by the HTML parser.
- When the parser encounters a new element that cannot be nested inside an open element, it may automatically close that element.
- Browsers may recover differently in malformed or invalid markup, particularly when CSS and JavaScript interact with the resulting DOM.
- Unexpected DOM structures can affect styling, spacing, accessibility, and element selection in scripts.

For example, a browser may implicitly close the heading when it encounters the paragraph. Therefore, the page might still appear to work, but relying on automatic error recovery is not a good development practice.

Analysis: Explicitly closing elements and following HTML5 standards produces predictable document structure and reduces compatibility problems.

3. Identify missing semantic elements

Semantic element	**Purpose**
<header>	Displays the company name and introductory content.

<nav>	Contains navigation links, when navigation is required.
<main>	Identifies the main content of the page.
<section>	Groups related information under a heading.
<footer>	Contains copyright, contact details, or related information.

Semantic elements provide meaning to the document rather than merely grouping content.

4. Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Company Website</title>

</head>

<body>

   <header>

      <h1>Welcome to Our Company</h1>

      <p>Delivering quality and innovation.</p>

   </header>


   <main>

      <section aria-labelledby="about-title">

         <h2 id="about-title">About Us</h2>

         <p>

            Our company provides innovative solutions
```

```
            to meet customer needs.
        </p>
    </section>
</main>

<footer>
    <p>&copy; 2026 Our Company. All rights reserved.</p>
</footer>
</body>
</html>
```

Why this version is better:

Includes the HTML5 doctype and required document structure.

Uses properly nested elements with explicit closing tags.

Applies semantic HTML5 elements to organize content.

Includes UTF-8 character encoding and responsive viewport settings.

Uses descriptive headings in a logical hierarchy.

Provides a predictable structure for CSS, JavaScript, and assistive technologies.

The structure is standards-compliant HTML5 and can be checked using the W3C Markup Validation Service.

5. Suggest techniques to ensure cross-browser compatibility

Validate HTML: Use the W3C validator to detect missing tags, invalid nesting, and markup errors.

Use CSS resets: Apply a consistent baseline for default margins, padding, and box sizing.

Follow web standards: Use valid HTML5 and standard CSS properties.

Test across browsers: Verify the page in Chrome, Firefox, Edge, and Safari.

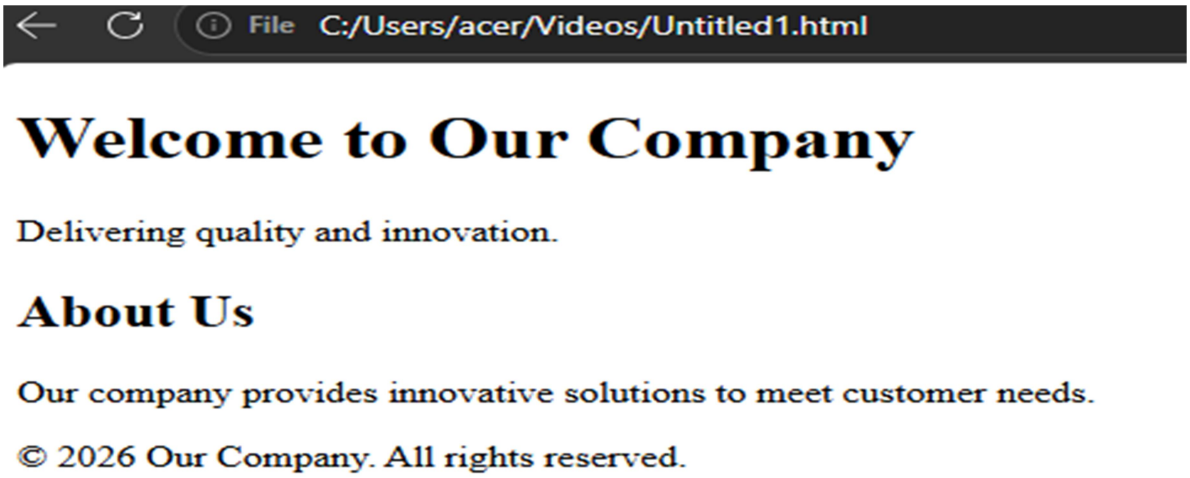
Use responsive design: Apply flexible layouts, media queries, and suitable viewport settings.

Avoid browser-specific code: Prefer standard APIs and use feature detection when necessary.

Test accessibility: Check keyboard navigation, heading hierarchy, and screen-reader compatibility.

Conclusion: Valid HTML5 markup, semantic elements, responsive CSS, and cross-browser testing ensure consistent rendering and improve website reliability.

OUTPUT:



Question 3: Form Submission Failure

1. Identify issues in the form configuration

The given form is:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

The main issues are:

- The button uses type="button" instead of type="submit".
- A button of type button does not automatically submit the form.
- The GET method places submitted form values in the URL query string.
- The password field could expose sensitive information in URLs, browser history, and logs if the form were submitted using GET.
- The form has no associated labels for accessibility.
- The inputs have no required attributes.
- The form does not indicate that secure HTTPS communication is necessary.
- The most direct reason for the submission failure is the incorrect button type.

2. Analyze why the HTTP request is not triggered

The browser normally initiates form submission when the user activates a submit button or when another supported form-submission action occurs.

In this example, the button is declared as:

<button type="button">Login</button>

This button is an ordinary clickable button. It does not submit the form automatically.

Consequently, clicking Login does not normally trigger a request to /submit.

If the button were changed to type="submit", the browser would submit the form using the configured GET method. However, GET would still be inappropriate for sending a password.

Key point: The button type causes the immediate failure, while the GET method introduces a separate security problem.

3. Interpret the role of the GET method in this scenario

The GET method requests a resource from the server. When used for an HTML form, the browser appends successful form-control values to the URL as query parameters.

For example, if the submit button were corrected but GET were retained, the request could resemble:

GET /submit?username=arun&password=abc123 HTTP/1.1

Host: example.com

This example illustrates why GET must not be used for password submission.

Limitations of GET for login forms:

Sensitive data may appear in URLs.

- URLs can be stored in browser history and server logs.
- URLs may be shared or exposed through other mechanisms.
- URLs have practical length limits.
- GET requests are intended primarily for retrieving resources, not performing a login action that changes authentication state.

GET is suitable for non-sensitive searches, filters, and other retrieval operations.

For login forms, POST over HTTPS is the appropriate choice. POST does not itself encrypt the data; HTTPS provides encryption in transit.

4. Propose a corrected implementation

The following implementation fixes the button issue, uses POST, and improves accessibility.

```
<!DOCTYPE html>

<html lang="en">

<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Secure Login</title>
</head>
<body>
    <main>
        <section aria-labelledby="login-title">
            <h1 id="login-title">User Login</h1>

            <form action="/submit" method="POST">
                <label for="username">Username:</label>
                <input
                    type="text"
                    id="username"
                    name="username"
                    autocomplete="username"
                    required
                >

                <br><br>

                <label for="password">Password:</label>
                <input
```

```
            type="password"
            id="password"
            name="password"
            autocomplete="current-password"
            required
          >

          <br><br>

          <button type="submit">Login</button>
        </form>
      </section>
    </main>
</body>
</html>
```

Explanation of the correction:

- method="POST" sends the submitted credentials in the HTTP request body.
- type="submit" triggers normal browser form submission.
- type="password" masks the password characters on screen.
- required ensures that both fields must be completed before normal browser submission.
- <label> improves accessibility.
- autocomplete helps browsers identify the username and current-password fields.

The server must still verify the credentials, manage authentication securely, and return appropriate success or error responses.

5. Suggest improvements for secure data transmission

- Use HTTPS: Encrypt communication between the browser and server using TLS.
- Use POST: Send credentials in the request body rather than the URL.
- Validate on the server: Treat browser-side validation as a usability aid, not a security boundary.
- Hash passwords securely: Store passwords using a suitable password-hashing algorithm such as Argon2id, scrypt, or bcrypt, with appropriate parameters.
- Implement session security: Use appropriately configured secure session cookies, including HttpOnly, Secure, and suitable SameSite settings.
- Prevent brute-force attacks: Apply rate limiting, monitoring, and suitable login protections.
- Avoid logging credentials: Never log passwords or expose them in error messages.
- Protect against CSRF where applicable: Use suitable anti-CSRF protections for authentication and session-related actions.

Conclusion: Correcting the submit button resolves the immediate form-submission issue. Using POST over HTTPS, validating credentials on the server, and applying secure authentication practices makes the login process more reliable and secure.

OUTPUT:

